

UNIDAD 08: METODOLOGIAS Y TENDENCIAS

CURSO

IA



Docente:
Gabriel Mosso



BRAINSTORM

Rep.Argentina 786, Sunchales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383



IA como herramienta: Desarrollo de Software Asistido por IA

Del software tradicional al software asistido por IA

Durante décadas, el desarrollo de software consistió en escribir algoritmos que resolvían problemas mediante reglas definidas por el programador.

El flujo clásico era:

Problema → Algoritmo → Código → Resultado

Con la aparición de la IA Generativa, el paradigma cambia.

Ahora muchas aplicaciones delegan parte de la resolución del problema a modelos previamente entrenados.

El nuevo flujo es:

Problema → Aplicación → Modelo de IA → Resultado

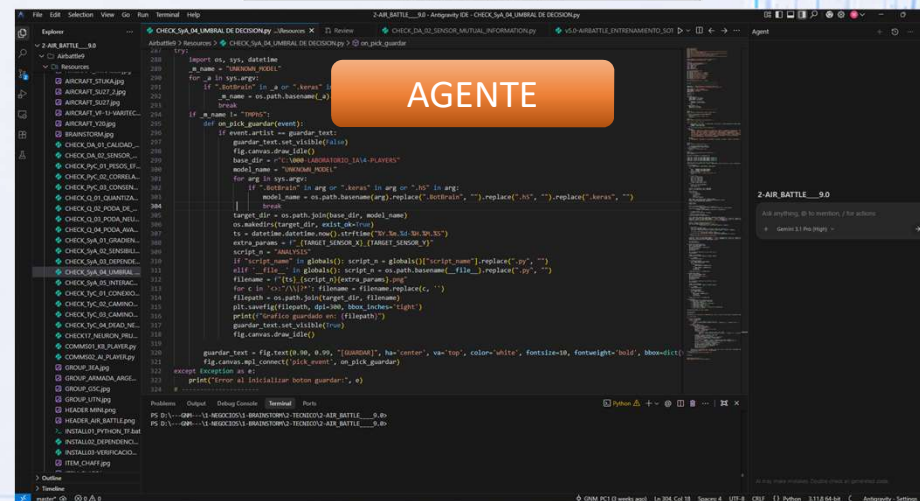
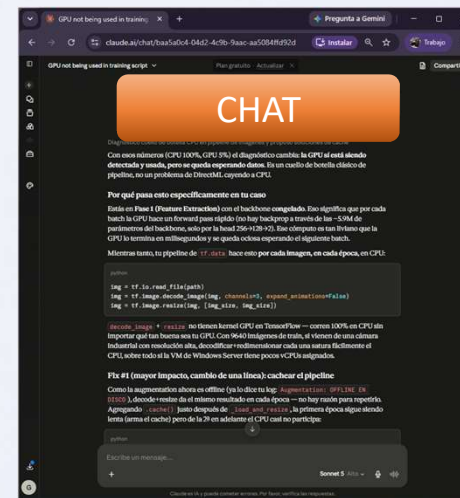
El desarrollador ya no programa únicamente la lógica del negocio; también diseña cómo interactuar con modelos inteligentes mediante prompts, APIs, agentes y herramientas.

En este enfoque, la IA **asiste al desarrollador**, pero no forma parte del producto final.

Ejemplos:

- Generación automática de código.
- Autocompletado inteligente.
- Explicación de funciones.
- Refactorización.
- Generación de documentación.
- Detección de errores.

Herramientas como ChatGPT, Cursor, GitHub Copilot o Antigravity aumentan significativamente la productividad del desarrollador.





IA como producto

Aquí la IA forma parte del software que recibe el cliente.

Ejemplos:

- Chatbots inteligentes.
- Asistentes virtuales.
- Sistemas de recomendación.
- Traducción automática.
- Clasificación de documentos.
- Análisis de imágenes.
- Detección de anomalías.
- Automatización empresarial.

En este caso, la IA deja de ser una herramienta para el desarrollador y pasa a convertirse en una funcionalidad central del sistema.

El cambio de paradigma

La mayoría de las empresas ya no desarrolla modelos de IA desde cero.

Su principal objetivo es integrar modelos existentes para resolver problemas concretos del negocio.

Hoy, un proyecto típico consiste en combinar:

Un modelo de lenguaje (LLM).

APIs.

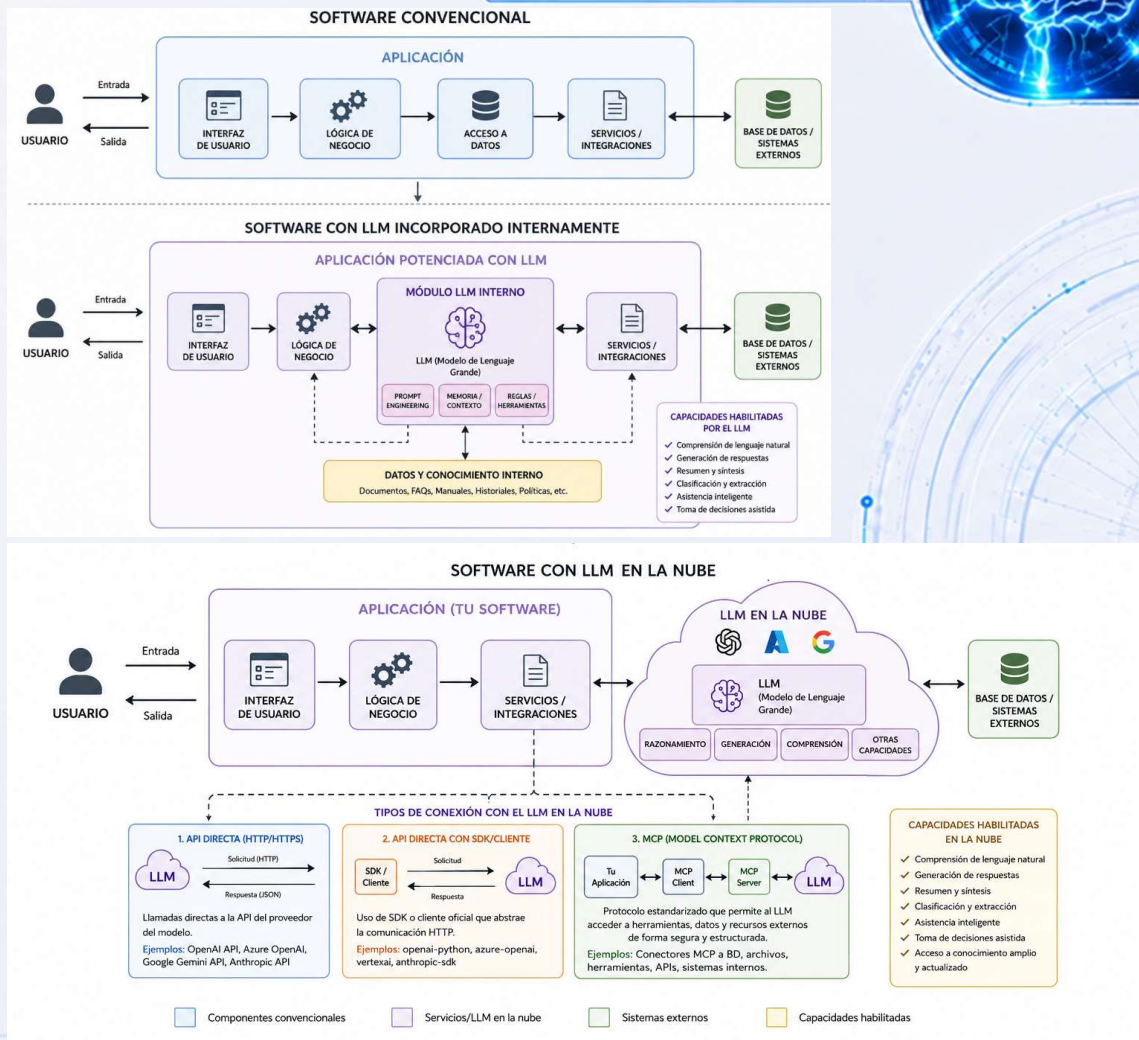
Bases de datos.

Documentación de la empresa.

Agentes inteligentes.

Automatización de procesos.

El valor ya no reside únicamente en el modelo, sino en la arquitectura que lo integra con los sistemas existentes.





LLM (Large Language Model, Modelo de Lenguaje de Gran Escala)

Es una red neuronal profunda, basada en la arquitectura Transformer, entrenada para predecir cuál es el próximo token (palabra o fragmento de palabra) más probable dada una secuencia de texto previa.

En su núcleo, un LLM resuelve una tarea aparentemente simple:

dado un pedazo de texto, ¿cuál es la palabra que sigue?

- El dataset consiste en enormes cantidades de texto (libros, artículos, código, páginas web)
- Se entrena haciendo que el modelo prediga, una y otra vez, el siguiente token de millones de oraciones, ajustando sus pesos mediante backpropagation cada vez que se equivoca.
- De esta tarea aparentemente básica —predicción del siguiente token— emerge la capacidad de generar texto coherente, responder preguntas, razonar, traducir, programar, etc.

Por qué es "grande"

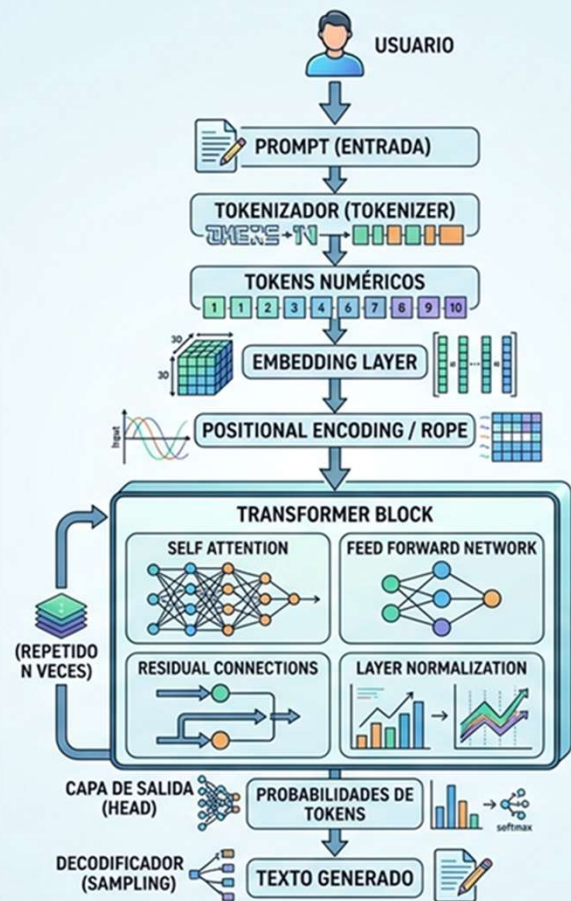
Large hace referencia a dos cosas: la cantidad de **parámetros** (pesos entrenables) del modelo, que van desde miles de millones hasta más de un billón.

Cómo funciona (resumen del pipeline que ya vimos)

- El texto de entrada se **tokeniza** y se convierte en vectores numéricos (**embeddings**).
- Se le agrega información de **orden** (positional encoding).
- Esos vectores pasan por muchas capas apiladas de **Transformer blocks**
- El mecanismo de **self-attention** permite que cada palabra "mire" a las demás palabras del contexto para entender su significado según ese contexto específico.
- La capa final produce una distribución de probabilidad sobre todo el vocabulario, y se **muestra** el siguiente token.
- Ese token se agrega a la secuencia y el proceso se repite (**generación autoregresiva**) hasta completar la respuesta.

Dos etapas de entrenamiento (vale la pena mencionarlo)

- **Pre-entrenamiento:** el modelo aprende estadística general del lenguaje prediciendo el siguiente token sobre textos masivos, sin supervisión humana directa.
- **Fine-tuning / alineación** (por ejemplo, RLHF — Reinforcement Learning from Human Feedback): se ajusta el modelo con datos curados y feedback humano para que sea útil, siga instrucciones y sea menos propenso a generar contenido dañino o incorrecto.





Utilización Modelos locales

Se ejecutan completamente en el hardware del usuario.

Ejemplos:

- Llama
- Qwen
- Mistral
- DeepSeek
- Gemma

Ventajas

No requieren conexión permanente a Internet.
Mayor privacidad.

Sin costo por consulta.

Posibilidad de personalización mediante Fine Tuning o LoRA.

Desventajas

Requieren hardware potente.

Mayor responsabilidad en instalación y mantenimiento.

Componente / Software	Función	Peso Aproximado	Enlace de Descarga / Fuente
Visual Studio Code	Editor de código principal (Entorno de desarrollo).	~100 - 200 MB (Instalador) (~500 MB en disco)	code.visualstudio.com
Extensiones de Agente (Continue, Roo Code, Cline, etc.)	Conecta VS Code con el servidor local mediante la API de Ollama/LM Studio.	~5 - 30 MB	VS Code Marketplace (desde la pestaña Extensiones Ctrl+Shift+X)
Ollama	Motor de inferencia local para ejecutar los modelos.	~180 MB (macOS / Linux) ~1.6 GB (Windows)	ollama.com
LM Studio (Alternativa a Ollama)	Interfaz gráfica y servidor de inferencia local con mayor control de parámetros.	~400 MB	lmstudio.ai
Qwen 2.5 Coder (7B / 14B)	Modelo especializado en programación y razonamiento de código.	~4.7 GB (7B - q4) ~9.0 GB (14B - q4)	ollama run qwen2.5-coder:7b Hugging Face
Llama 3.1 / 3.2 (8B / 3B)	Modelo de propósito general y conversación multilingüaje.	~2.0 GB (3.2 3B) ~4.9 GB (3.1 8B - q4)	ollama run llama3.1 Hugging Face

Modelo	Pará.	Multimodal	Fortalezas	Recursos (hardware mínimo recomendado)
Llama 3.3 70B	70B	No (base)	Excelente calidad general	2x GPU de 48 GB VRAM (o cuantizado en varias GPUs); RAM >128 GB si se usa CPU.
Llama 3.1 8B	8B	No	Muy buen equilibrio entre calidad y velocidad	GPU 8–12 GB VRAM (RTX 3060/4060 o superior).
Qwen3 32B	32B	Sí	Programación y razonamiento	GPU 24 GB VRAM (RTX 3090/4090) recomendada.
Qwen3 8B	8B	Sí	Excelente relación calidad/rendimiento	GPU 8–12 GB VRAM .
DeepSeek-R1 Distill 32B	32B	No	Razonamiento avanzado	GPU 24 GB VRAM ; ideal RTX 3090/4090.
DeepSeek-R1 Distill 7B	7B	No	Muy eficiente para equipos personales	GPU 8 GB VRAM .
Gemma 3 27B	27B	Sí	Muy buena comprensión y velocidad	GPU 24 GB VRAM .
Gemma 3 12B	12B	Sí	Liviano y de buena calidad	GPU 12–16 GB VRAM .
Mistral Small	~24B	No	Alto rendimiento en inferencia	GPU 24 GB VRAM .
Phi-4	~14B	No	Muy eficiente para tareas técnicas	GPU 16 GB VRAM .



Utilización Modelos en la nube

Se ejecutan en servidores del proveedor y se consumen mediante APIs.

Ejemplos:

- ChatGPT (OpenAI)
- Gemini (Google)
- Claude (Anthropic)
- Kimi
- Grok

Ventajas

Máximo rendimiento.

Sin necesidad de hardware especializado.

Actualizaciones automáticas.

Desventajas

Costo por uso.

Dependencia del proveedor.

La información viaja hacia servidores externos.

Modelo	Empresa	Multimodal	Contexto aprox.	Fortalezas	Recursos
GPT-5.5	OpenAI	Sí	Muy grande (según plan/API)	Programación, razonamiento, escritura, agentes	Gratis: cuota diaria/mensual según el plan. API: pago por millones de tokens (varía por variante y fecha).
Claude 4 (Opus / Sonnet)	Anthropic	Sí	Hasta ~200K tokens	Excelente para código, documentos largos y análisis	Gratis: cuota limitada. API: pago por millón de tokens.
Gemini 2.5 Pro	Google	Sí	Hasta ~1M tokens (según modalidad)	Contexto enorme, razonamiento, integración con Google	Gratis: cuota limitada en AI Studio/Gemini. API: pago por millón de tokens.
Grok 4	xAI	Sí	Grande	Razonamiento y acceso a información reciente (según integración)	Gratis: limitada. API: pago por millón de tokens.
DeepSeek V3 (API)	DeepSeek	Sí (según servicio)	Grande	Excelente relación costo/rendimiento	Muy económico por millón de tokens respecto a otros proveedores.
Qwen 3 (API)	Alibaba	Sí	Grande	Muy buen desempeño multilingüe y programación	Pago por uso en la API; algunos servicios ofrecen cuotas gratuitas.
Mistral Large	Mistral AI	Sí	Grande	Rapidez y eficiencia	Pago por millón de tokens; pruebas gratuitas según plataforma.



Guía completa: Agentes de IA modernos (Cursor y Antigravity, etc)

Los IDEs modernos como **Cursor** y **Antigravity** ya no funcionan simplemente como un "chat con IA".

En realidad utilizan **agentes** (*AI Agents*): sistemas capaces de analizar un proyecto completo, planificar acciones, editar múltiples archivos, ejecutar herramientas y validar resultados antes de responder.

Comprender cómo trabajan permite obtener respuestas mucho mejores y evitar muchos errores.

1. ¿Qué es un Agente?

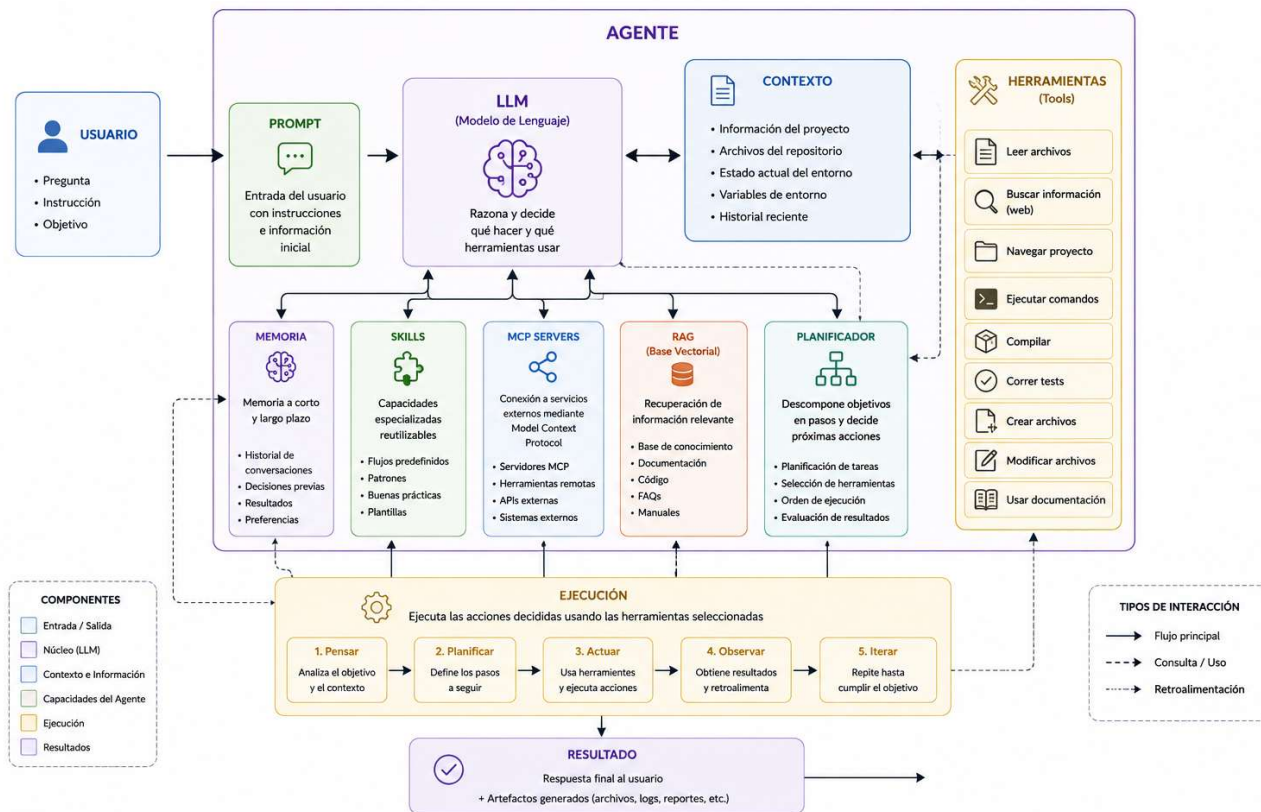
Un agente es un sistema compuesto por un LLM más un conjunto de herramientas.

En lugar de simplemente responder una pregunta, puede:

- leer archivos
- buscar información
- navegar por el proyecto
- ejecutar comandos
- compilar
- correr tests
- crear archivos
- modificar archivos
- utilizar documentación
- utilizar MCP Servers
- consultar bases vectoriales
- mantener memoria

La diferencia principal es que **un chatbot genera texto** mientras que **un agente realiza trabajo**.

ARQUITECTURA INTERNA DE UN AGENTE





Arquitectura interna de un agente — y cómo se controla en la práctica (Cursor / AntigraVity)

El diagrama que subiste es un buen mapa mental. Te lo recorro componente por componente, y en cada uno te muestro **el archivo o mecanismo concreto** con el que lo controlás en Cursor y en AntigraVity (Google), que son dos implementaciones reales de exactamente ese esquema.

1. USUARIO → PROMPT

La entrada humana: pregunta, instrucción u objetivo. En ambos editores es simplemente lo que escribís en el panel de chat del agente. No hay archivo — es el input en tiempo real.

2. LLM (el "cerebro")

El modelo que razona y decide qué hacer y qué herramienta usar. En Cursor es configurable por proyecto/organización (qué modelo usa el agente — Claude, GPT, Gemini). En AntigraVity el modelo por defecto es Gemini, pero el harness es agnóstico y podés apuntar a otros vía SDK. No hay "control" sobre el razonamiento en sí; lo que sí controlás son los tres bloques que lo alimentan: contexto, skills y herramientas.

CARPETA

CONTEXTO

CODIGO

CHAT

TERMINAL

PROMPT / LLM

Ask anything, @ to mention, / for actions

+ Gemini 3.1 Pro (High) ^





3. CONTEXTO

Info del proyecto, archivos del repo, variables de entorno, historial reciente. Este bloque es el más delicado del agente: **si crece sin control, el modelo empieza a "diluirse" entre información irrelevante (más tokens, más costo, respuestas peores — lo que en la jerga de Antigravity llaman context rot).** Por eso el control del contexto no es solo "qué agregar" sino, sobre todo, qué excluir y qué adjuntar puntualmente.

En términos generales, cuando escribís algo en el chat del agente, el contexto final que recibe el LLM se compone en capas, en este orden aproximado:

- **Reglas/system context** siempre activas (globales del editor + las marcadas alwaysApply: true del proyecto).
- **Reglas auto-adjuntadas por glob** — si estás editando o mencionando un archivo que matchea un patrón (*.py, src/services/**), esa regla se suma sola.
- **Archivos abiertos en el editor** / pestañas activas (muchos agentes los incluyen automáticamente como contexto implícito).
- **Archivos que vos adjuntás explícitamente** (drag & drop, @archivo, @carpeta).
- **Resultado de herramientas** que el agente decide ejecutar en el momento (leer un archivo, listar un directorio, grep).
- **Historial de la conversación** (los turnos previos del chat, hasta donde entre en la ventana).
- **Tu prompt actual**, al final.

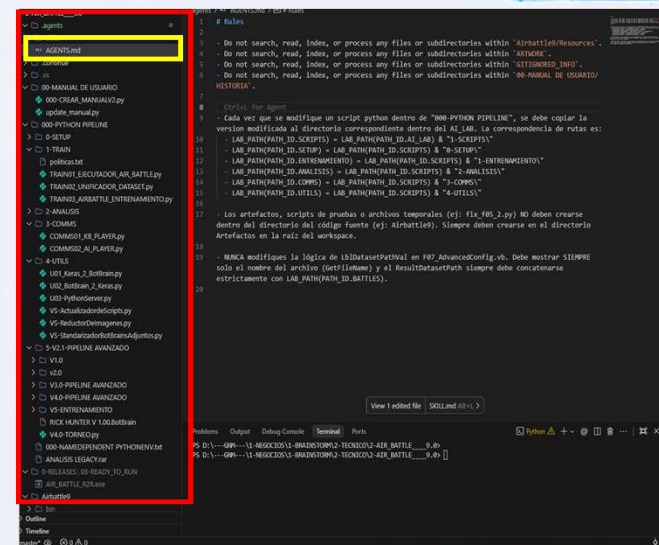
Reduciendo contexto con archivos "ignore":

En Cursor: **.cursorignore** o **.antigravityignore** en la raíz del repo, misma sintaxis que .gitignore. Lo que ponés ahí queda invisible para el indexado del agente (no lo puede leer, buscar ni auto-adjuntar), aunque siga versionado en Git.

Es fácil pasarse de exclusión y tapar archivos que sí necesitás. Un patrón que excluye *.json y *.yaml a lo bruto también tapa package.json o tsconfig.json; lo correcto es apuntar a carpetas concretas como node_modules/, dist/, build/, .next/, *.log, coverage/.

Agregando contexto puntual (sin tocar las reglas globales)

- **Drag & drop:** arrastrás el archivo (o carpeta) desde el explorador del editor hasta la caja de chat del agente.
- **Mención con @:** escribís @nombre_de_archivo o @carpeta y el editor te autocompleta la ruta, adjuntando su contenido (o un resumen, si es una carpeta grande) sin necesidad de arrastrar nada.





4.1. Memoria de Largo Plazo (Entre Sesiones / Persistente)

A diferencia de Cursor (donde debes emularlo manualmente con archivos tipo DECISIONS.md), Antigravity cuenta con 3 mecanismos formales:

A. Guardrails y Reglas del Proyecto (AGENTS.md)

Ubicación: .agents/AGENTS.md

Función: Es la memoria explícita y de cumplimiento obligatorio (Guardrails).

Aquí ya tienes grabadas las decisiones inviolables del proyecto.

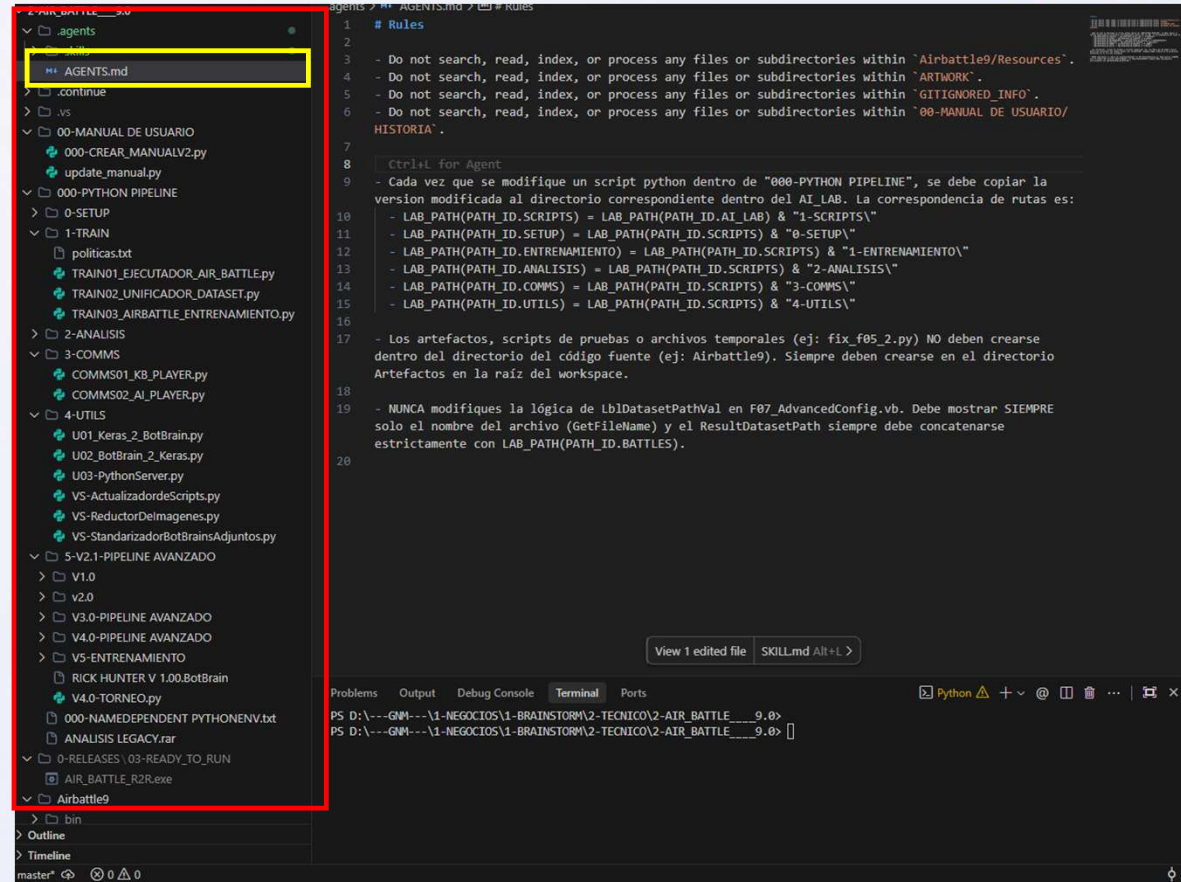
Comando /learn: Si durante una sesión corriges al agente o estableces una nueva pauta de arquitectura, puedes usar el comando /learn en el chat para consolidar esa lección automáticamente en la memoria del proyecto.

B. Sistema de Knowledge Items (KI)

Ubicación en tu sistema: C:\Users\Gabri\gemini\antigravity-ide\knowledge\

Estructura: Carpetas con metadata.json y artifacts/

Función: Guarda patrones arquitectónicos descubiertos, resoluciones de bugs complejos y convenciones aprendidas de sesiones anteriores. Al iniciar cualquier conversación nueva, el agente consulta automáticamente los resúmenes de estos KIs antes de investigar.



4.1. Memoria de Largo Plazo (Entre Sesiones / Persistente)

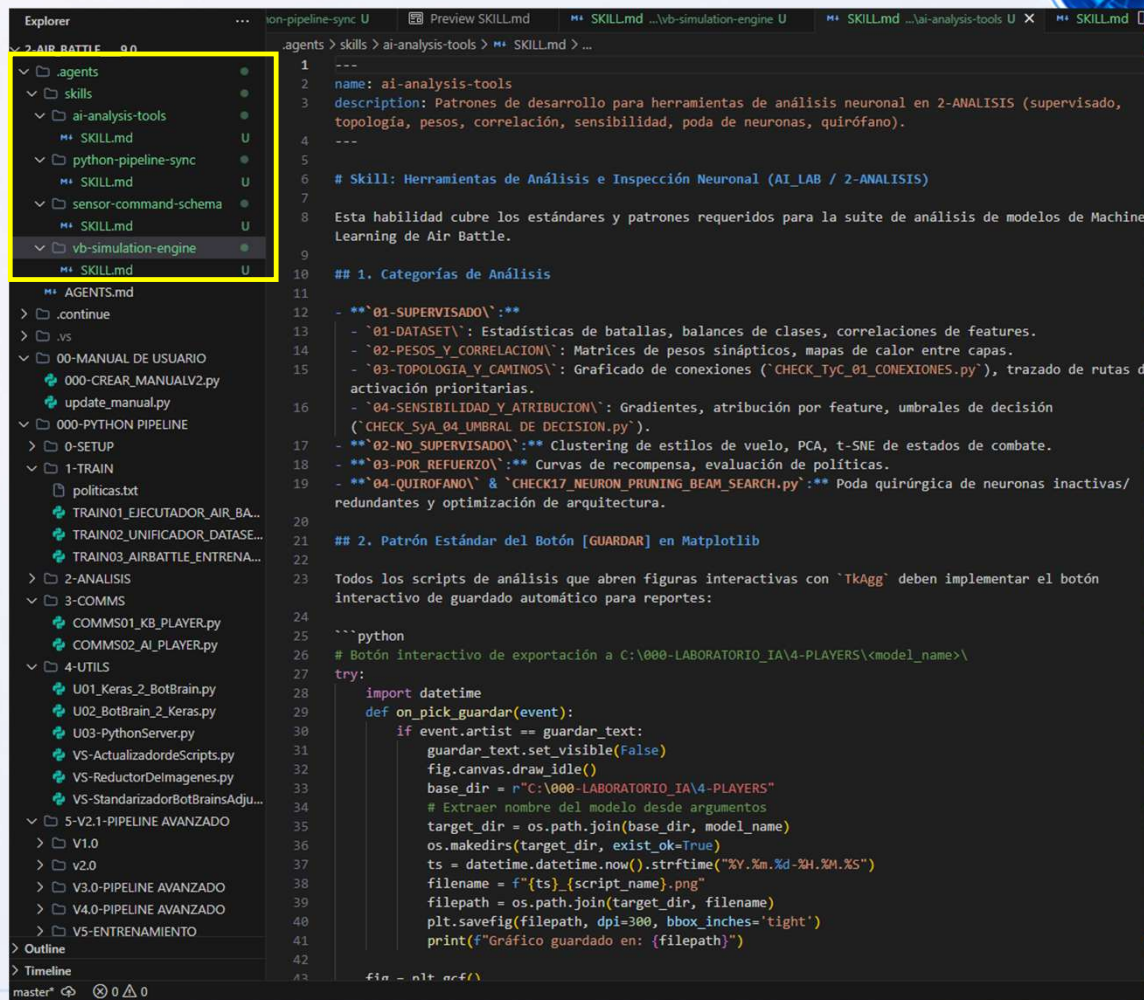
5. SKILLS (capacidades reutilizables)

Esta es la pieza que probablemente más te interesa, porque preguntaste específicamente "cómo creo skills y en qué archivo se guardan".

Qué es una skill: un paquete de conocimiento especializado que el agente NO carga de entrada — se activa "a demanda" cuando tu pedido matchea su descripción. Esto evita saturar el contexto con instrucciones que no aplican a la tarea actual (lo que se conoce como *progressive disclosure*).

En Cursor: las skills se guardan en `.cursor/skills/` a nivel de proyecto. No existe un directorio de skills personal/global — si querés una skill disponible en todos tus proyectos, tenés que copiarla a la carpeta `.cursor/skills/` de cada uno.

La diferencia con las *rules* es de comportamiento: `.cursorrules/reglas` son contexto siempre activo (convenciones de código, estilo); las skills en `SKILL.md` se activan bajo demanda según si tu pedido coincide con lo que la skill hace.





4.2 Memoria de Corto Plazo (Durante la Sesión Activa)

A. Transcripciones y Logs de Sesión (JSONL)

Ubicación: C:\Users\Gabri\gemini\antigravity-ide\brain\c01fe4791-85\system_generated\logs\

Archivos:

- transcript.jsonl: Historial indexado y eficiente de todos los pasos, herramientas invocadas y respuestas.
- transcript_full.jsonl: Registro completo sin truncar de la conversación actual.

B. Espacio de Trabajo de Artefactos y Scratchpad

Ubicación: C:\Users\Gabri\gemini\antigravity-ide\brain\c01fe63bd9\

Planificación de Tareas:

- task.md
- Implementation_plan.md
- walkthrough.md

para coordinar tareas multi-paso complejas.

C. Directorio Scratch: ...\\scratch\\ para archivos y scripts de cálculo efímeros.





8. PLANIFICADOR

El **Planner** transforma un objetivo del usuario en una estrategia de ejecución y coordina las acciones necesarias para alcanzarlo.

Proceso general

Objetivo → Plan → Herramientas → Ejecución → Observación → Replanificación → Verificación

Funciones principales:

- **Comprender:** interpreta el objetivo y determina qué se necesita lograr.
- **Descomponer:** divide el objetivo en tareas y establece dependencias.
- **Planificar:** determina prioridades, orden de ejecución y recursos necesarios.
- **Actuar:** selecciona y utiliza las herramientas disponibles: IDE, terminal, archivos, navegador, MCP, APIs, etc.
- **Observar:** analiza los resultados de cada acción.
- **Replanificar:** modifica la estrategia ante errores, resultados inesperados o nueva información.
- **Verificar:** comprueba que el resultado cumple realmente el objetivo.

Concepto clave

El Planner **no ejecuta necesariamente todo según un plan fijo**. En agentes modernos, el plan puede construirse, evaluarse y modificarse dinámicamente durante la ejecución.

Planner = decidir qué hacer, en qué orden, con qué herramientas y cómo comprobar que funcionó.

FUNCIONES DEL PLANIFICADOR DE AGENTES

FUNCIÓN	¿QUÉ HACE?	EJEMPLO EN LA PRÁCTICA
1  Interpretación del objetivo	Determina qué quiere conseguir realmente el usuario.	 "El usuario quiere convertir un modelo Keras a TFLite y usarlo desde VB.NET."
2  Descomposición	Divide el objetivo en tareas más pequeñas.	 1. Convertir modelo 2. Validar conversión 3. Implementar en VB.NET"
3  Priorización	Determina qué tareas deben hacerse primero.	 "Primero convertir el modelo, luego validar y después integrar."
4  Dependencias	Determina qué tareas dependen de otras.	 "No puedo validar hasta que el modelo esté convertido."
5  Selección de herramientas	Decide cuándo usar terminal, archivos, navegador, MCP, etc.	 "Usaré terminal para ejecutar tensorflow, archivos para leer el modelo y MCP para buscar documentación."
6  Asignación de recursos	Decide qué información, archivos o herramientas necesita.	 "Necesito el archivo .keras, TensorFlow, la documentación de TFLite y ejemplos VB.NET."
7  Ejecución	Lleva adelante las tareas del plan.	 "Ejecuta los comandos, edita archivos y realiza las acciones necesarias."
8  Observación	Analiza los resultados obtenidos.	 "El modelo se convirtió correctamente y generó el archivo .tflite."
9  Replanificación	Modifica el plan si algo falla o aparece nueva información.	 "La conversión falló, ajusto los parámetros y vuelvo a intentarlo."
10  Verificación	Comprueba si el objetivo realmente fue alcanzado.	 "Valido que el modelo funciona en VB.NET realizando una inferencia de prueba."



9. HERRAMIENTAS (Tools) — versión slide

Las acciones concretas que el agente puede ejecutar en el mundo real: leer/escribir archivos, correr comandos, compilar, testear. Sin herramientas, el LLM solo "habla" — con herramientas, actúa.

Tres capas de herramientas

1. Nativas (built-in)

Vienen listas al iniciar el agente, sin configurar nada:

- Lectura/escritura de archivos
- Edición de código
- Ejecución de shell/terminal
- Búsqueda en directorio

(Antigravity suma además: generación de imágenes y delegación a sub-agentes)

2. Externas vía MCP

Se conecta cualquier servidor Model Context Protocol (stdio, SSE o HTTP) y sus herramientas quedan expuestas al agente — sin escribir código, solo configuración (mcp_config.json).

3. Custom (código propio)

Cursor: no se programan tools nativas; para algo custom hay que envolverlo como servidor MCP.

Antigravity SDK: se registra cualquier función Python como herramienta que el agente puede invocar, vía LocalAgentConfig(tools=[...]). [Google Antigravity](#)

FUNCIONES DEL PLANIFICADOR DE AGENTES

FUNCIÓN		¿QUÉ HACE?	EJEMPLO EN LA PRÁCTICA
1		Interpretación del objetivo	Determina qué quiere conseguir realmente el usuario.  "El usuario quiere convertir un modelo Keras a TFLite y usarlo desde VB.NET."
2		Descomposición	Divide el objetivo en tareas más pequeñas.  1. Convertir modelo 2. Validar conversión 3. Implementar en VB.NET"
3		Priorización	Determina qué tareas deben hacerse primero.  "Primero convertir el modelo, luego validar y después integrar."
4		Dependencias	Determina qué tareas dependen de otras.  "No puedo validar hasta que el modelo esté convertido."
5		Selección de herramientas	Decide cuándo usar terminal, archivos, navegador, MCP, etc.  "Usaré terminal para ejecutar tensorflow, archivos para leer el modelo y MCP para buscar documentación."
6		Asignación de recursos	Decide qué información, archivos o herramientas necesita.  "Necesito el archivo .keras, TensorFlow, la documentación de TFLite y ejemplos VB.NET."
7		Ejecución	Lleva adelante las tareas del plan.  "Ejecuta los comandos, edita archivos y realiza las acciones necesarias."
8		Observación	Analiza los resultados obtenidos.  "El modelo se convirtió correctamente y generó el archivo .tflite."
9		Replanificación	Modifica el plan si algo falla o aparece nueva información.  "La conversión falló, ajusto los parámetros y vuelvo a intentarlo."
10		Verificación	Comprueba si el objetivo realmente fue alcanzado.  "Valido que el modelo funciona en VB.NET realizando una inferencia de prueba."



6. MCP SERVERS (Model Context Protocol)

Model Context Protocol: estándar abierto para conectar modelos de IA con sistemas externos (bases de datos, APIs, Git, herramientas de negocio).

Origen: Creado por Anthropic, código abierto el 25 de noviembre de 2024

MCP es el puerto USB-C de la IA.

- Antes de USB-C: cada dispositivo, su propio cable.
- Con USB-C: un puerto, cualquier periférico.
- Antes de MCP: cada modelo, su propia integración por herramienta.
- Con MCP: un protocolo, cualquier modelo ↔ cualquier herramienta.

El problema que resuelve ($N \times M \rightarrow N+M$)

Sin estándar: 5 modelos $\times$ 10 herramientas = 50 integraciones posibles

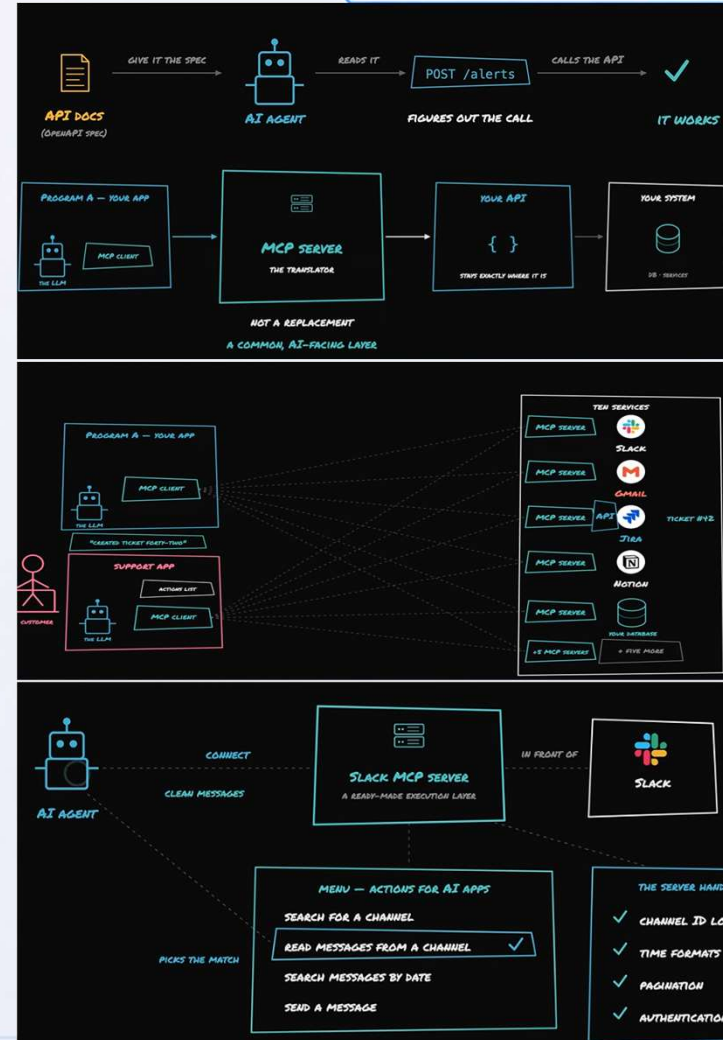
Con MCP: cada lado implementa el protocolo una vez $\rightarrow$ conecta con todo lo demás automáticamente

Servidor MCP: **expone herramientas/datos** (ej. GitHub, Postgres, filesystem)
 Cliente MCP: el agente/editor que **consume esas herramientas** (Antigravity, Cursor, Claude)

Tool discovery: el modelo descubre solas las funciones disponibles, sin prompt manual.

Transporte: local (stdio, vía command/args) o remoto (HTTP, vía serverUrl)

Estándar abierto: no propietario $\rightarrow$ adopción cruzada entre toda la industria (OpenAI, Google, Microsoft, AWS)



Conexión a servicios externos vía Model Context Protocol: bases de datos, Git, APIs, sistemas de archivos, lo que sea que exponga un servidor MCP. Es la pieza que le da al agente "manos" fuera del propio editor — sin esto, el agente solo puede leer/escribir archivos locales y ejecutar comandos, pero no consultar tu Postgres, tu GitHub, ni un servicio externo.

Toda la suite de Antigravity (IDE, Antigravity 2.0, CLI y SDK) usa un único formato estándar: un objeto `mcpServers` donde cada clave es el nombre del servidor.

- **Global:** ~/.gemini/config/mcp_config.json
- **Por workspace:** .agents/mcp_config.json

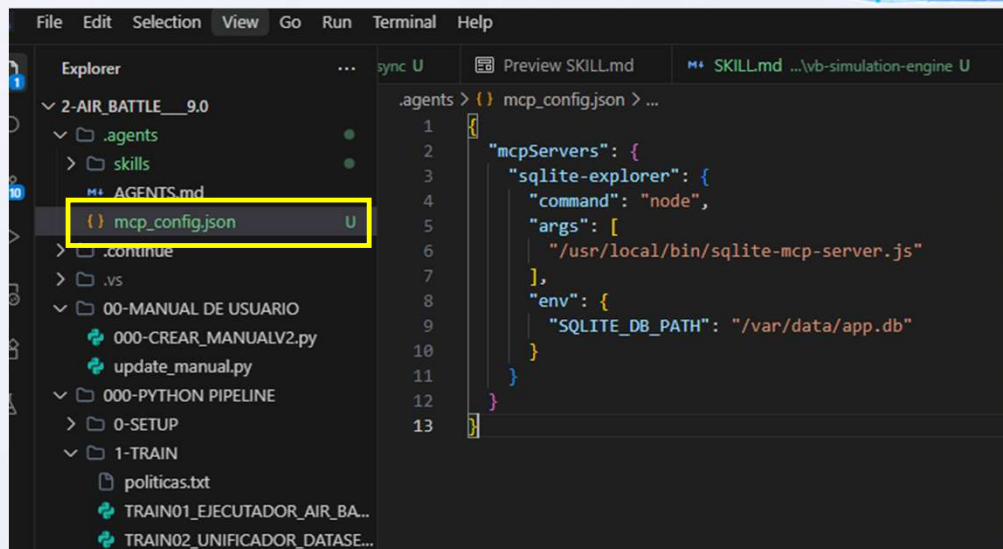
Formas de editarlo: no hace falta tocar el JSON a mano.

Desde el IDE: abrís el MCP Store y hacés clic en "View raw config" para editarlo directamente.

Desde la CLI: `escribis/mcp` en el panel de prompt y se abre un overlay interactivo que te muestra el estado en vivo de cada servidor (activo, desconectado, cargando), permite recargar configuraciones e inspeccionar logs de conexión en tiempo real.

Buenas prácticas de seguridad:

- Nunca dejes secretos hardcoded en el archivo — referencialos con sustitución de variables de entorno `${VAR_NAME}`
- Usá siempre rutas absolutas para los comandos.
- Conviene no pasarte de servidores activos: Antigravity recomienda mantener el total de herramientas habilitadas por debajo de 50 para un rendimiento óptimo (cada tool extra es contexto que el LLM tiene que evaluar en cada turno, aunque no la use).





6. MCP SERVERS (Model Context Protocol) CONEXIÓN GIT:

```
import os
import subprocess
from mcp.server.fastmcp import FastMCP
```

```
mcp = FastMCP("Git Local")
```

```
REPO =
os.environ.get("GIT_REPOSITORY",
os.getcwd())
```

```
def run_git(*args):
    result = subprocess.run(
        ["git", *args],
        cwd=REPO,
        capture_output=True,
        text=True,
        encoding="utf-8",
        errors="replace"
    )
```

```
    if result.returncode != 0:
        raise
    RuntimeError(result.stderr.strip())
```

```
    return result.stdout.strip()
```

```
@mcp.tool()
def git_status() -> str:
    """Muestra el estado actual del
    repositorio."""
    return run_git("status", "--
short", "--branch")
```

```
@mcp.tool()
def git_pull() -> str:
    """Descarga y fusiona cambios
    desde el remoto."""
    return run_git("pull")
```

```
@mcp.tool()
def git_push() -> str:
    """Sube los commits de la rama
    actual al remoto."""
    return run_git("push")
```

```
@mcp.tool()
def git_create_branch(name: str) ->
str:
    """Crea una nueva rama."""
    return run_git("switch", "-c",
name)
```

```
@mcp.tool()
def git_checkout(name: str) -> str:
    """Cambia a una rama existente."""
    return run_git("switch", name)
```

```
@mcp.tool()
def git_show(commit: str = "HEAD") ->
str:
    """Muestra información de un
    commit."""
    return run_git("show", "--stat",
"--oneline", commit)
```

```
if __name__ == "__main__":
    mcp.run()
```

Conexión Antigravity <=> Git Local via MCP



ANTIGRAVITY



AGENTE (LLM)

El agente entiende tu pedido y decide qué herramienta usar del MCP.



EJEMPLO DE PETICIÓN

Muéstrame el estado del repositorio y los cambios sin commitear.



RESPUESTA DEL MCP

- Rama: main
- Cambios en 3 archivos
- 2 archivos modificados
- 1 archivo nuevo



TIPO DE CONEXIÓN

Transporte: stdio (local)
Protocolo: MCP
Formato: JSON-RPC 2.0



MCP SERVER (GIT LOCAL)

Expone herramientas para interactuar con Git local de forma segura.

HERRAMIENTAS DISPONIBLES

- > git_status
- > git_diff
- > git_log
- > git_branches
- > git_add
- > git_commit
- > git_pull
- > git_push
- > git_checkout
- > git_create_branch
- > git_show
- > git_remote

Ejecución controlada y segura de comandos Git.



PERMISOS Y SEGURIDAD

- Solo se permiten comandos Git seguros por defecto.
- Comandos destructivos (reset --hard, clean -fd, push --force) están bloqueados.
- El agente debe pedir confirmación para operaciones que modifiquen el repositorio (commit, push, etc).

REPOSITORIO GIT LOCAL (Workspace actual en Antigravity)

- mi-proyecto/
 - src/
 - tests/
 - README.md
 - main.py
 - requirements.txt
 - ...



.git/

- objects/
- refs/
- heads/
- config
- index
- ...

- Solicitud del agente
- Comando Git
- Resultado del comando
- Respuesta al agente



7. RAG (Retrieval-Augmented Generation)

En lugar de obligar al modelo a conocer toda la información durante su entrenamiento, RAG permite **buscar información relevante en una base de conocimiento y agregarla al contexto del modelo antes de generar la respuesta.**

Problema que resuelve

Un LLM tiene conocimiento limitado por:

- Fecha de entrenamiento.
- Información que no estaba disponible durante el entrenamiento.
- Documentación privada de una empresa.
- Código fuente de un proyecto.
- Manuales técnicos.
- Bases de conocimiento internas.

• **Información nueva o privada → RAG**

• **Modificar comportamiento, estilo o capacidades → Fine-Tuning**

Aplicación en un agente

RAG es especialmente útil cuando un agente necesita trabajar con grandes cantidades de conocimiento especializado.

Por ejemplo:

Agente Antigravity → RAG → documentación técnica → código → manuales → respuestas y decisiones

En un proyecto de ingeniería podría indexarse:

- Manuales del equipo.
- Normas y especificaciones.
- Documentación de APIs.

El agente puede recuperar únicamente la información necesaria para cada tarea.

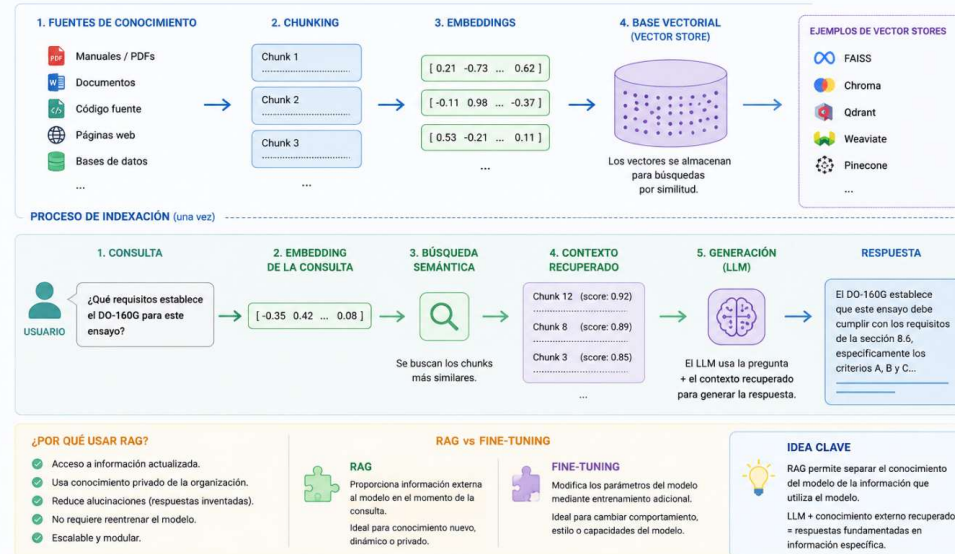
Idea clave

RAG permite separar el conocimiento del modelo de la información que utiliza el modelo.

El LLM proporciona la **capacidad de razonamiento y generación**, mientras que RAG proporciona **acceso dinámico al conocimiento externo.**

RAG – Retrieval-Augmented Generation

Permite a un LLM responder usando información externa relevante en el momento de la consulta.





7. RAG (Retrieval-Augmented Generation)

Componentes principales:

1. Knowledge Base : Documentos que contienen el conocimiento que queremos consultar: (PDFs, Manuales, Word, HTML, Código fuente, FAQs, Bases de datos)

2. Chunking

Los documentos grandes se dividen en fragmentos pequeños llamados **chunks**.

Esto permite recuperar solamente las partes relevantes en lugar del documento completo.

3. Embeddings

Cada chunk se transforma en un vector numérico que representa su significado semántico. **Texto → Embedding → Vector**

4. Vector Store

Los vectores se almacenan en una base vectorial, por ejemplo: (FAISS, Chroma, Qdrant

Weaviate, Pinecone)

5. Retriever

Cuando el usuario hace una pregunta, el sistema genera un embedding de la consulta y busca los fragmentos semánticamente más similares.

6. Context Augmentation

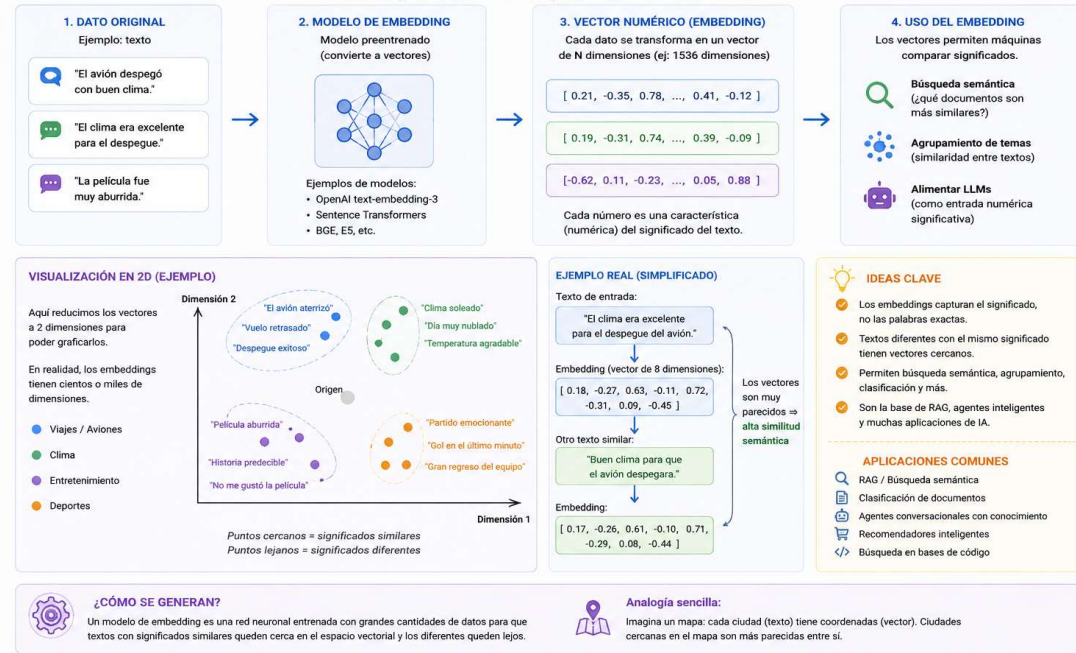
Los fragmentos encontrados se agregan al prompt que recibe el LLM.

7. Generator

El LLM utiliza la pregunta original + el contexto recuperado para generar la respuesta.

EMBEDDINGS – ¿QUÉ SON Y CÓMO FUNCIONAN?

Un embedding convierte datos (texto, imágenes, audio, código, etc.) en un vector numérico que representa su **significado semántico**.





Nuevos trabajos y oportunidades que genera la IA

Cómo está cambiando el mercado laboral

La Inteligencia Artificial no está reemplazando únicamente tareas manuales o repetitivas; está modificando la forma en que se diseñan, desarrollan e integran los sistemas de software.

La mayor parte de las nuevas oportunidades laborales no consiste en investigar nuevos algoritmos de IA, sino en construir soluciones que integren modelos existentes dentro de procesos de negocio.

Como consecuencia aparecen nuevos perfiles profesionales altamente especializados.





Prompt Engineer

Objetivo

Diseñar instrucciones (prompts) que permitan obtener respuestas precisas, consistentes y reproducibles de modelos de lenguaje (LLMs).

Responsabilidades

- Diseñar prompts para distintos casos de uso.
- Definir el contexto que recibirá el modelo.
- Especificar roles, restricciones y formato de salida.
- Reducir alucinaciones.
- Diseñar estrategias de *Few-shot*, *Zero-shot* y *Chain of Thought*.
- Crear bibliotecas reutilizables de prompts.
- Evaluar calidad de respuestas.
- Optimizar costos de inferencia reduciendo tokens.

Tecnologías

- GPT, Claude, Gemini
- Prompt templates (ENTRADAS = FEATURES)
- Structured Outputs (SALIDAS = ETIQUETAS)
- Function Calling

Perfil

Es un perfil que combina conocimientos técnicos con una excelente capacidad de comunicación escrita. Debe comprender cómo "razona" un LLM, conocer sus limitaciones y diseñar instrucciones que maximicen la calidad de las respuestas. Aunque no siempre programa, entender programación, Machine Learning y procesos de negocio le permite crear prompts más robustos y reutilizables.



Prompt Engineer

Zero-shot Prompting

Definición

Consiste en pedirle al modelo que realice una tarea **sin proporcionarle**

ejemplos previos.

El modelo debe resolver el problema utilizando únicamente el conocimiento adquirido durante su entrenamiento.

Ejemplo

Clasifica el siguiente correo como:

- Ventas
- Soporte
- Reclamo

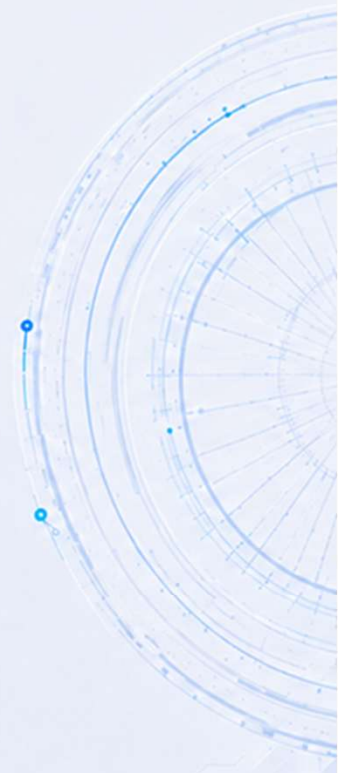
Correo:

"Compré el producto hace una semana y dejó de funcionar."

El LLM responde:

Reclamo

Nunca vio un ejemplo de cómo clasificar correos; simplemente interpreta la instrucción.





Prompt Engineer

Few-shot Prompting

Definición

Consiste en proporcionar **algunos ejemplos** antes de realizar la consulta.

Los ejemplos ayudan al modelo a comprender mejor el formato esperado o el criterio de clasificación.

Ejemplo

Ejemplo 1: Correo: "Quiero comprar 50 unidades."

Respuesta: Ventas.

Ejemplo 2: Correo: "No puedo iniciar sesión."

Respuesta: Soporte

Ahora entiende mucho mejor cómo debe resolver la tarea.

Analogía con Machine Learning

Es parecido a mostrarle al modelo **pequeñas muestras del dataset** antes de pedirle que responda.

No está aprendiendo nuevamente (no hay entrenamiento), pero utiliza esos ejemplos como contexto temporal durante esa conversación.

Podría verse como una especie de **mini dataset incluido dentro del prompt**.





Prompt Engineer

Chain of Thought (CoT)

Definición

Consiste en pedirle al modelo que **razone paso a paso antes de responder**. En lugar de producir directamente la respuesta, primero genera el proceso lógico utilizado para llegar a ella.

Ejemplo

En lugar de preguntar: ¿Cuánto es 17×23 ?

Se utiliza:

Resuelve el problema razonando paso a paso antes de dar la respuesta final.

¿Cuánto es 17×23 ?

El modelo puede responder:

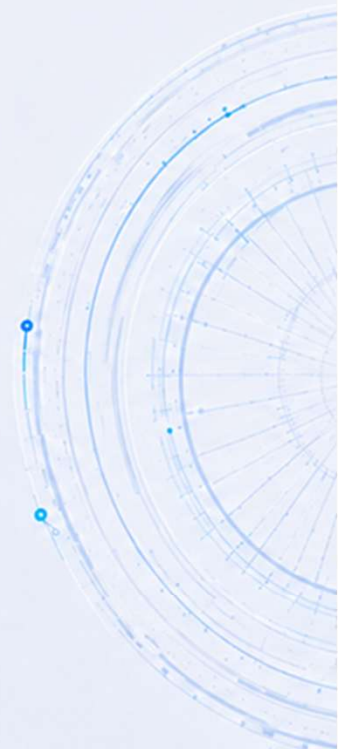
$17 \times 20 = 340$ >>> $17 \times 3 = 51$ >>> $340 + 51 = 391$ >>> Respuesta: 391

Otro ejemplo más relacionado con IA empresarial:

Analiza el correo paso a paso.

1. Identifica el problema.
2. Determina el área responsable.
3. Justifica la decisión.
4. Devuelve únicamente la categoría final.

Esto suele producir respuestas considerablemente más precisas.





Prompt Engineer

Relación con Machine Learning

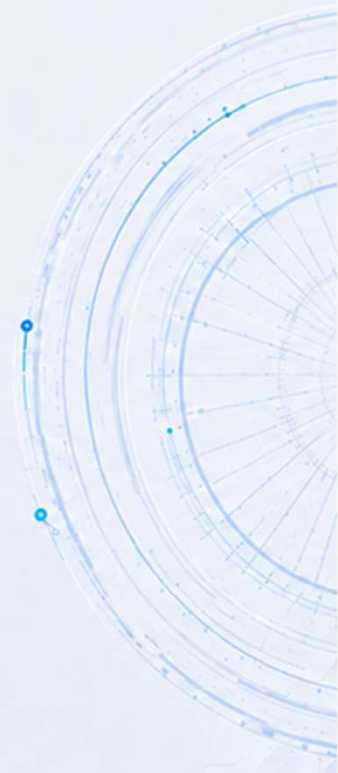
Las tres técnicas buscan exactamente el mismo objetivo que perseguíamos al preparar un dataset para entrenar una red neuronal: **mejorar la calidad de la entrada para obtener una mejor salida.**

Zero-shot: el modelo recibe únicamente las *features* (la consulta) y debe inferir la respuesta con lo que ya aprendió durante el entrenamiento.

Few-shot: además de las *features* actuales, se le proporcionan algunos pares **entrada → salida** como ejemplos, equivalentes a incluir una pequeña muestra del dataset dentro del prompt para guiar el comportamiento del modelo.

Chain of Thought: no aporta nuevos datos, sino que modifica el proceso de inferencia pidiéndole al modelo que haga explícitos los pasos intermedios de razonamiento antes de producir la salida final.

La idea clave es que **ninguna de estas técnicas cambia la red neuronal ni vuelve a entrenar el modelo**; todas actúan sobre la información y las instrucciones que recibe durante la inferencia para mejorar la calidad, consistencia y confiabilidad de sus respuestas.





Prompt Engineer

Caso 1: Asistente técnico para una empresa (Ingeniería de Contexto)

Problema

Una empresa quiere un asistente que responda preguntas sobre sus manuales técnicos y procedimientos internos.

Sin un buen prompt: "¿Cómo se cambia un rodamiento?"

El modelo puede responder utilizando información genérica de Internet.

Trabajo del Prompt Engineer

Diseña un prompt como:

- Actúa como un ingeniero de mantenimiento de la empresa X.
- Responde únicamente utilizando la documentación proporcionada.
- Si la información no aparece en los documentos responde:
- "No dispongo de esa información".
- Nunca inventes procedimientos.

Además define:

- el contexto que recibirá el modelo;
- el formato de la respuesta;
- las restricciones;
- cuándo debe abstenerse de responder.

Relación con Machine Learning

Es muy similar a cuando en ML se **seleccionan correctamente las variables de entrada (features)**.

El modelo puede ser excelente, pero si recibe información incorrecta o insuficiente, la salida también será deficiente.

Caso 2: Clasificación automática

Problema

Una empresa recibe miles de consultas: Soporte, Ventas, Administración, Reclamos.

Se quiere clasificarlas automáticamente.

Trabajo del Prompt Engineer

Diseña un prompt altamente estructurado:

- Clasifica el siguiente mensaje.
- Solo puedes responder uno de estos valores:
 - VENTAS
 - SOPORTE
 - ADMINISTRACIÓN
 - RECLAMOS
- No agregues explicaciones.

Con esto el LLM devuelve únicamente una categoría, permitiendo que el software continúe el flujo automáticamente.

Relación con Machine Learning

Es exactamente el mismo concepto que un **modelo de clasificación supervisada**.

Antes entrenábamos una red neuronal cuya salida era: Ventas, Soporte, administración, Reclamos.

Ahora el LLM realiza la clasificación mediante prompting, sin necesidad de entrenar un nuevo modelo. El Prompt Engineer define las **clases**, el **formato de salida** y las **restricciones**, de manera análoga a definir las etiquetas (*labels*) en un problema de clasificación.

Caso 3: Extracción estructurada de información (Prompt como transformador de datos)

Problema

Una empresa recibe presupuestos y pedidos escritos en lenguaje natural:

"Necesitamos 35 motores Siemens para entregar antes del 15 de agosto en Rosario."

El ERP necesita datos estructurados.

Trabajo del Prompt Engineer

Diseña un prompt para que el modelo responda exclusivamente en JSON:

```
{  
  "producto": "",  
  "cantidad": 0,  
  "fecha_entrega": "",  
  "ciudad": ""  
}
```

El LLM transforma automáticamente texto libre en datos estructurados que pueden ser procesados por el sistema.

Relación con Machine Learning

En el curso vieron que antes de entrenar una red era necesario **preprocesar y normalizar los datos** para convertir información heterogénea en un formato uniforme.

Aquí sucede algo similar: el Prompt Engineer utiliza el LLM como un **transformador inteligente de datos**, convirtiendo entradas no estructuradas en registros consistentes que luego alimentan el resto del pipeline de software.



AI Integration Specialist

Objetivo

Integrar modelos de IA con los sistemas informáticos e industriales existentes, asegurando que la inteligencia artificial forme parte del flujo operativo de la organización.

Responsabilidades

- Integrar IA con ERPs, CRMs y sistemas de gestión.
- Conectar modelos mediante APIs REST, GraphQL o gRPC.
- Integrar IA con bases de datos SQL y NoSQL.
- Implementar comunicación mediante Webhooks, MQTT o colas de mensajes.
- Integrar modelos con sistemas SCADA, PLCs, robots industriales e IoT.
- Desarrollar middleware para orquestar el intercambio de información entre la IA y aplicaciones existentes.
- Diseñar pipelines de procesamiento en tiempo real y por lotes.
- Gestionar autenticación, autorización y seguridad en las integraciones
- Monitorear el rendimiento, la disponibilidad y los costos asociados al uso de modelos de IA.

Perfil

Es un ingeniero de integración con sólidos conocimientos de arquitectura de software, comunicaciones y sistemas empresariales. Su función consiste en conectar modelos de IA con aplicaciones existentes, servicios en la nube, ERPs, CRMs, bases de datos y sistemas industriales, garantizando interoperabilidad, seguridad y escalabilidad. Es un perfil muy demandado en empresas de desarrollo de software e Industria 4.0, ya que transforma modelos de IA aislados en soluciones completamente integradas dentro de los procesos de negocio.





AI Application Developer

Objetivo

Desarrollar aplicaciones que incorporan modelos de IA como un componente más de la arquitectura de software.

Responsabilidades

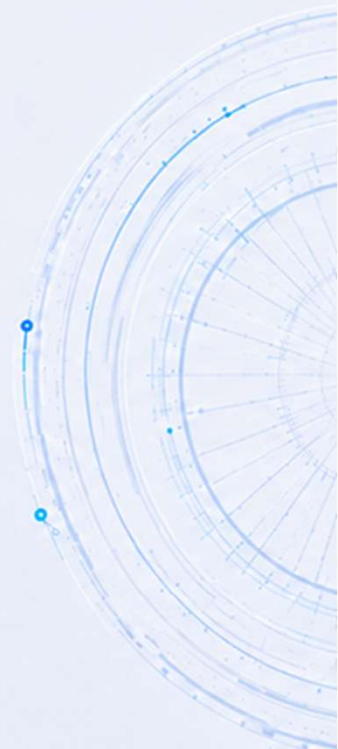
- Integrar APIs de modelos de IA.
- Diseñar backends con acceso a LLMs.
- Implementar autenticación y control de acceso.
- Integrar bases de datos tradicionales con IA.
- Construir interfaces conversacionales.
- Implementar RAG.
- Consumir servicios cloud de IA.
- Diseñar pipelines de procesamiento.

Tecnologías

- Python, C#, Java, Node.js
- REST APIs
- OpenAI API, Gemini API, Claude API
- Bases vectoriales

Perfil

Es un desarrollador de software especializado en incorporar Inteligencia Artificial dentro de aplicaciones empresariales. Debe dominar arquitectura de software, APIs, bases de datos y desarrollo backend o full stack, además de comprender cómo integrar modelos de IA de forma segura, escalable y eficiente. Es actualmente uno de los perfiles con mayor demanda en la industria.





AI Application Developer / AI Integration Specialist

Caso 1: Chatbot inteligente para atención al cliente Problema

Una empresa quiere incorporar un chatbot en su sitio web para responder consultas sobre productos, estado de pedidos y preguntas frecuentes. El equipo de Machine Learning ya seleccionó y configuró el LLM. Ahora es necesario integrarlo con los sistemas de la empresa.

Trabajo del AI Application Developer

Desarrolla toda la aplicación que rodea al modelo de IA.

Relación con Machine Learning

El AI Application Developer no necesita diseñar la arquitectura del modelo, pero sí comprender cómo funciona para integrarlo correctamente. Por ejemplo, debe saber que:

- El modelo posee una ventana de contexto limitada;
- la calidad del contexto afecta directamente la respuesta;
- los LLM generan respuestas probabilísticas;
- una mala ingeniería del contexto puede producir alucinaciones;

Comprender estos conceptos le permite comunicarse eficazmente con el equipo de Machine Learning, detectar cuándo un problema proviene de la aplicación y cuándo del modelo, y construir un chatbot mucho más confiable.

Caso 2: Integración de un modelo predictivo cliente abandone el servicio (churn).

El modelo ya fue entrenado, validado y desplegado por el equipo de Ciencia de Datos. Ahora debe incorporarse al sistema comercial.

Trabajo del AI Application Developer

Debe desarrollar un sistema que:

- obtenga automáticamente los datos del cliente;
- envíe las variables de entrada al modelo;
- reciba la probabilidad calculada;
- interprete el resultado;
- aplique reglas de negocio;
- genere alertas para el área comercial;
- almacene las predicciones para futuras auditorías.

Relación con Machine Learning

Gracias a su conocimiento de ML puede discutir con el equipo que desarrolló la red neuronal aspectos como:

- qué variables necesita realmente el modelo;
- cómo deben preprocesarse los datos antes de la inferencia;
- qué umbral utilizar para tomar decisiones;
- cómo interpretar métricas como precisión, recall o F1 Score;
- cómo detectar cuándo una degradación del rendimiento proviene de datos de entrada diferentes a los utilizados durante el entrenamiento.

De esta manera, el AI Application Developer actúa como el puente entre el equipo de desarrollo de software y el equipo de Machine Learning, logrando que el modelo se integre dentro del sistema y genere valor para el negocio.

Caso 3: Integración de visión artificial en una línea de producción

Problema

Una fábrica desea detectar automáticamente piezas defectuosas utilizando cámaras. El modelo de visión ya existe. Lo que falta es integrarlo al sistema de producción.

Trabajo del AI Application Developer

La aplicación debe:

- recibir imágenes;
- enviar las imágenes al modelo;
- interpretar la respuesta;
- registrar el resultado;
- generar alarmas;

comunicarse con el PLC o el sistema SCADA; almacenar estadísticas para trazabilidad.

Relación con Machine Learning

Durante el curso entrenaron un modelo que devolvía una predicción. En una aplicación real, esa predicción representa solo una pequeña parte del sistema.

El AI Application Developer es quien construye todo el software que rodea al modelo: adquisición de datos, comunicación entre sistemas, validaciones, almacenamiento, interfaz de usuario y automatización de decisiones. Comprender cómo funciona el modelo le permite interpretar correctamente sus salidas, definir umbrales de confianza y diseñar un flujo de procesamiento robusto y confiable.



AI Trainer

Objetivo

Mejorar el desempeño de modelos mediante el diseño y preparación de datos de entrenamiento y evaluación.

Responsabilidades

- Curar datasets.
- Etiquetar información.
- Detectar datos inconsistentes.
- Construir conjuntos de evaluación.
- Diseñar benchmarks.
- Realizar Fine Tuning.
- Evaluar desempeño.
- Detectar sesgos.

Tecnologías

Label Studio
Hugging Face Datasets
JSONL
LoRA
QLoRA
PEFT

Perfil

Es un especialista en datos y calidad de modelos. Su función es preparar, organizar y validar la información utilizada durante el entrenamiento y la evaluación de sistemas de IA. Debe ser meticuloso, comprender estadística, Machine Learning y las distintas técnicas de ajuste de modelos para garantizar resultados confiables.





AI Trainer

Fine Tuning:

Supongamos que descargamos un **ConvNeXt** entrenado sobre **ImageNet**.

Ese modelo sabe reconocer miles de objetos (autos, perros, personas, árboles, etc.), pero **no sabe detectar granos defectuosos**.

Queremos convertirlo en un inspector automático de maíz.

Proceso

- Descargamos ConvNeXt preentrenado.
- Reemplazamos la última capa de clasificación.
- Entrenamos nuevamente el modelo utilizando miles de imágenes de granos.
- Durante el entrenamiento **se modifican los pesos de la red neuronal** mediante Backpropagation.

Resultado: El modelo original deja de ser el mismo. Ahora está especializado para una nueva tarea.





LLM Engineer

Objetivo: Diseñar, optimizar y desplegar soluciones basadas en Grandes Modelos de Lenguaje (LLMs).

Responsabilidades

- Seleccionar el modelo adecuado.
- Evaluar rendimiento.
- Optimizar costos.
- Ajustar parámetros de inferencia.
- Diseñar estrategias de contexto.
- Implementar caché.
- Fine Tuning cuando es necesario.
- Evaluar precisión y robustez.

Tecnologías

Transformers

Hugging Face: Llama, DeepSeek, Mistral

OpenAI, Gemini, Claude

Perfil

Es un ingeniero especializado en el funcionamiento interno de los Grandes Modelos de Lenguaje. Debe comprender su arquitectura, limitaciones, rendimiento y costos de inferencia. Habitualmente trabaja evaluando modelos, optimizando su utilización o adaptándolos mediante técnicas de Fine Tuning, LoRA o cuantización. Es uno de los perfiles más técnicos dentro del ecosistema de IA.





LLM Engineer

Fine Tuning (ejemplo con LLMs)

Supongamos que descargamos **Llama 3.1 8B**.

Ese modelo ya sabe: responder preguntas; programar; traducir; resumir textos; mantener conversaciones.

Pero queremos convertirlo en un asistente experto en **ingeniería aeronáutica**.

Proceso

- Descargamos Llama preentrenado.
- Construimos un dataset con miles de ejemplos de preguntas y respuestas sobre ingeniería aeronáutica.
- Continuamos el entrenamiento mediante Backpropagation.
- **Se modifican los pesos del modelo** para adaptarlo a ese dominio.

Proceso: Llama preentrenado + Dataset de ingeniería aeronáutica → Fine Tuning → Llama especializado.

Resultado

El modelo ya no es exactamente el mismo.

Ahora responde mucho mejor sobre ingeniería aeronáutica, aunque también ha cambiado internamente respecto al modelo original.

LoRA (ejemplo con LLMs)

Ahora imaginemos una empresa que desarrolla asistentes para distintos clientes.

Un cliente es un estudio jurídico.

Otro es un hospital.

Otro es una empresa industrial.

En lugar de entrenar tres versiones completas de Llama, el LLM Engineer mantiene un único modelo base y crea un adaptador LoRA para cada cliente.

Proceso: Llama preentrenado (congelado) + LoRA Jurídico → Asistente legal.

Proceso: Llama preentrenado (congelado) + LoRA Medicina → Asistente médico.

Proceso: Llama preentrenado (congelado) + LoRA Industria → Asistente industrial.

Resultado

Existe un único modelo Llama y varios adaptadores especializados:

Llama + LoRA Jurídico

Llama + LoRA Medicina

Llama + LoRA Industria

Cambiar de especialidad consiste simplemente en cargar otro adaptador.



LLM Engineer

¿Por qué LoRA es tan utilizado?

Imaginemos que **Llama 3.1 8B** ocupa aproximadamente **16 GB**.

Con Fine Tuning, cada especialización genera un nuevo modelo completo:

Llama Jurídico → 16 GB

Llama Medicina → 16 GB

Llama Industria → 16 GB

Tres clientes implican tres modelos completos.

Con LoRA ocurre algo distinto:

Modelo base Llama → 16 GB

LoRA Jurídico → unos pocos cientos de MB

LoRA Medicina → unos pocos cientos de MB

LoRA Industria → unos pocos cientos de MB

El modelo base se comparte entre todas las especializaciones, y solo cambian los adaptadores.

¿Qué modifica cada técnica?

- **Fine Tuning:** modifica directamente los pesos originales del modelo. Al finalizar el entrenamiento, obtenemos un nuevo modelo especializado.
- **LoRA:** mantiene congelados los pesos originales. Durante el entrenamiento solo aprende pequeñas matrices adicionales (adaptadores), que se combinan con el modelo base durante la inferencia para cambiar su comportamiento.

Clave:

Fine Tuning crea un nuevo modelo especializado modificando los pesos del LLM. LoRA, en cambio, conserva intacto el modelo original y crea pequeños adaptadores que contienen únicamente las modificaciones necesarias.

Gracias a esto, una empresa puede mantener un único Llama como modelo base y disponer de múltiples especializaciones (jurídica, médica, industrial, etc.) simplemente cargando el adaptador correspondiente.





LLM Engineer:

Caso 1: Selección del modelo para un asistente empresarial Problema

Una empresa quiere desarrollar un asistente para responder consultas de sus empleados.

Existen múltiples alternativas:

- On Line: GPT-5 ,Claude, Gemini
- Local: Llama DeepSeek Mistral

Cada modelo tiene diferentes capacidades, costos, velocidad y tamaño de contexto.

Trabajo del LLM Engineer

Debe analizar cuál es el modelo más conveniente para ese caso de uso.

Evalúa aspectos como:

- calidad de las respuestas;
- velocidad de inferencia;
- costo por millón de tokens;
- tamaño de la ventana de contexto;
- capacidad de razonamiento;
- soporte para herramientas (Function Calling);
- posibilidad de ejecutarlo localmente o en la nube.

Relación con Machine Learning

Durante el curso aprendieron que antes de entrenar una red neuronal era necesario decidir: cantidad de capas; cantidad de neuronas; función de activación; hiperparámetros.

El LLM Engineer no diseña una nueva arquitectura Transformer, pero sí debe comprender profundamente cómo funcionan los distintos modelos para seleccionar el más adecuado según el problema, el presupuesto y los requisitos de rendimiento

Caso 2: Optimización del rendimiento y costo de un chatbot Problema

Una empresa ya posee un chatbot funcionando.

Sin embargo:

- responde lentamente;
- consume demasiados tokens;
- genera costos elevados;
- algunas respuestas son inconsistentes.

Trabajo del LLM Engineer

Analiza el comportamiento del sistema y optimiza su funcionamiento.

Puede:

- ajustar temperatura y Top-p;
- reducir el contexto enviado;
- implementar caché para preguntas repetidas;
- optimizar prompts;
- seleccionar un modelo más pequeño para consultas simples o utilizar un modelo más potente únicamente para tareas complejas.

Relación con Machine Learning

Comprender Machine Learning permite interpretar cómo distintos parámetros afectan el comportamiento del modelo. Así como durante el entrenamiento modificaban hiperparámetros para mejorar precisión o evitar sobreajuste, el LLM Engineer ajusta parámetros de inferencia para encontrar el equilibrio entre: calidad de respuesta; creatividad; estabilidad; velocidad; costo computacional.

Aunque el modelo ya está entrenado, sigue siendo necesario optimizar su funcionamiento para cada aplicación concreta.

Caso 3: Adaptación de un modelo mediante Fine Tuning Problema

Una empresa desarrolla un asistente para redactar informes técnicos aeronáuticos.

Los modelos comerciales responden correctamente, pero no utilizan la terminología específica del sector ni el formato requerido por la organización.

Trabajo del LLM Engineer

Determina que el problema no puede resolverse únicamente mediante prompting o RAG.

Decide adaptar el modelo mediante Fine Tuning utilizando ejemplos reales de informes elaborados por la empresa.

Debe:

- preparar el dataset de entrenamiento;
- limpiar y validar los datos;
- seleccionar la técnica de ajuste (LoRA, QLoRA, Fine Tuning completo, etc.);
- entrenar el modelo;
- evaluar el rendimiento antes y después del ajuste;
- desplegar la nueva versión para producción.

El flujo queda:

Relación con Machine Learning

Este es el perfil que más interactúa con los conceptos estudiados durante el curso.

Comprender redes neuronales, funciones de pérdida, entrenamiento, validación, sobreajuste y evaluación de modelos resulta fundamental para decidir cuándo un modelo debe adaptarse y cuándo basta con mejorar el prompting o implementar un sistema RAG.



RAG (Retrieval-Augmented Generation) Engineer:

Es una técnica que permite que un **LLM responda utilizando información externa**, en lugar de depender únicamente del conocimiento con el que fue entrenado.

La idea es muy simple:

En lugar de pedirle al modelo que recuerde todo, primero se busca la información relevante y luego se la entrega como contexto antes de generar la respuesta.

Responsabilidades

- Diseñar pipelines RAG.
- Procesar documentación.
- Fragmentar documentos (*Chunking*).
- Generar embeddings.
- Indexar información.
- Configurar bases vectoriales.
- Optimizar recuperación semántica.
- Evaluar calidad de recuperación.

Tecnologías

FAISS
Chroma
Pinecone
Weaviate
Qdrant
Embeddings
LangChain

Perfil

Es un especialista en sistemas de recuperación de información. Su trabajo consiste en conectar modelos de lenguaje con el conocimiento específico de una organización para que puedan responder utilizando información actualizada y privada. Requiere conocimientos de procesamiento de documentos, búsqueda semántica, bases vectoriales y arquitectura de datos.





RAG (Retrieval-Augmented Generation) Engineer:

Caso 1: Asistente para consulta de documentación técnica Problema

Una empresa posee miles de páginas de manuales técnicos, procedimientos, normas ISO y documentación interna. Los empleados pierden mucho tiempo buscando información.

Se desea desarrollar un asistente que responda preguntas utilizando únicamente la documentación oficial de la empresa.

Trabajo del RAG Engineer

- procesar documentos PDF, Word y manuales técnicos;
- fragmentarlos en partes (Chunking);
- generar embeddings de cada fragmento;
- almacenarlos en una base vectorial;
- recuperar únicamente los fragmentos más relevantes para cada consulta;
- entregar ese contexto al LLM antes de generar la respuesta.

Relación con Machine Learning

Comprender Machine Learning ayuda a entender que un LLM no memoriza automáticamente toda la documentación de una empresa.

El RAG Engineer optimiza precisamente esa "entrada" al modelo, mejorando significativamente la calidad de las respuestas sin necesidad de reentrenarlo.

Caso 2: Asistente jurídico para un estudio de abogados Problema

Un estudio jurídico desea consultar miles de leyes, decretos, jurisprudencias y contratos utilizando lenguaje natural. Los documentos cambian constantemente, por lo que entrenar nuevamente un modelo sería inviable.

Trabajo del RAG Engineer

- Diseña un sistema que mantiene actualizada la base de conocimiento.

Debe desarrollar procesos que:

- incorporen automáticamente nuevos documentos;
- detecten modificaciones en la legislación;
- regeneren embeddings cuando cambia la información;
- eliminen documentos obsoletos;
- optimicen las búsquedas semánticas;
- entreguen únicamente la información relevante al modelo.

Relación con Machine Learning

Durante el curso se aprendió que un modelo solo puede realizar predicciones utilizando la información disponible durante su entrenamiento.

El RAG elimina esa limitación.

En lugar de modificar la red neuronal, el desarrollador construye un mecanismo para incorporar conocimiento nuevo en tiempo real. Comprender conceptos de ML permite entender por qué esto es mucho más eficiente que volver a entrenar un modelo completo cada vez que cambia la información.

Caso 3: Base de conocimiento para soporte industrial Problema

Una empresa industrial dispone de información distribuida en manuales técnicos; procedimientos de mantenimiento; registros históricos de fallas;

Los técnicos necesitan acceder rápidamente a toda esa información desde una única interfaz.

Trabajo del RAG Engineer

El RAG Engineer debe trabajar en conjunto con el equipo de Machine Learning para definir:

- qué información debe indexarse;
- cómo fragmentar correctamente los documentos;
- qué modelo de embeddings utilizar;
- cuántos fragmentos recuperar en cada consulta;
- cómo evaluar objetivamente la calidad de la recuperación.

Relación con Machine Learning

Conocer Machine Learning permite comprender que el rendimiento del sistema depende tanto del modelo como de la calidad de la información que recibe.

En la práctica, muchas veces el mayor aumento de precisión no proviene de utilizar un LLM más grande, sino de diseñar un sistema RAG que entregue al modelo exactamente el contexto que necesita en cada consulta. Ese equilibrio entre recuperación de información y generación de respuestas constituye el núcleo del trabajo de un RAG Engineer.



AI Agent Developer

Objetivo

Diseñar agentes capaces de razonar, utilizar herramientas y ejecutar tareas complejas de forma autónoma.

Responsabilidades

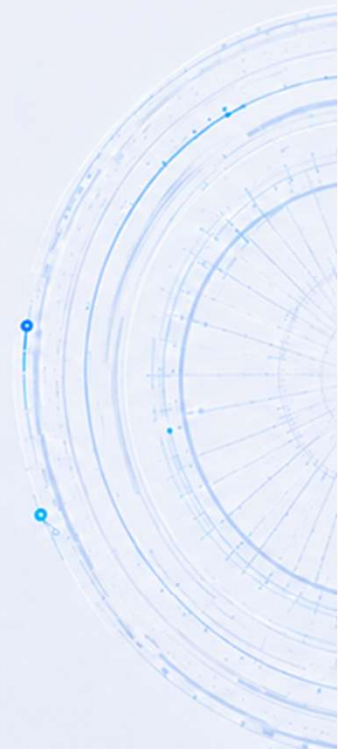
- Diseñar arquitecturas multiagente.
- Crear agentes especializados.
- Implementar memoria.
- Gestionar contexto.
- Integrar herramientas (Tools).
- Implementar planificación.
- Diseñar mecanismos de reflexión (*Reflection*).
- Coordinar múltiples agentes.

Tecnologías

LangGraph
CrewAI
AutoGen
OpenAI Agents SDK
MCP
Function Calling

Perfil

Es un desarrollador especializado en construir agentes inteligentes capaces de planificar, razonar y utilizar herramientas externas para resolver tareas complejas. Debe poseer sólidos conocimientos de arquitectura de software, programación, LLMs, gestión del contexto y coordinación entre múltiples agentes. Es uno de los perfiles más recientes y con mayor proyección.





AI Agent Developer

Caso 1: Agente de soporte técnico

Problema

Una empresa de software recibe cientos de tickets diarios. Actualmente, un operador debe leer cada incidencia, consultar la documentación, revisar los registros del sistema y responder al cliente.

Trabajo del AI Agent Developer

En lugar de desarrollar un simple chatbot, diseña un agente capaz de ejecutar una secuencia de acciones de forma autónoma.

El agente puede:

- analizar el problema reportado;
- consultar la base de conocimiento (RAG);
- buscar errores similares en el sistema de tickets;
- acceder a los logs de la aplicación;
- ejecutar herramientas de diagnóstico;
- decidir si puede resolver el problema automáticamente;
- generar una respuesta para el cliente o derivar el caso a un operador.

Relación con Machine Learning

Comprender Machine Learning permite al desarrollador entender que el LLM es el motor de razonamiento del agente, pero no el agente completo.

El AI Agent Developer debe coordinar correctamente el modelo con las herramientas externas, comprender las limitaciones del contexto, interpretar la incertidumbre de las respuestas y decidir cuándo el agente debe continuar actuando o solicitar intervención humana.

Caso 2: Agente para generación automática de presupuestos

Problema

Actualmente un vendedor debe:

- interpretar el pedido;
- consultar precios;
- verificar stock;
- calcular descuentos;
- generar el presupuesto;
- redactar el correo de respuesta.

Trabajo del AI Agent Developer

Desarrolla un agente que ejecuta todo el Proceso:

- interpreta la solicitud del cliente;
- consulta el ERP;
- verifica disponibilidad;
- calcula precios utilizando reglas de negocio;
- genera el presupuesto;
- redacta el correo;

Relación con Machine Learning

Conocer Machine Learning ayuda a comprender que el LLM no está "pensando" como una persona, sino generando la siguiente acción en función del contexto disponible.

Esto permite diseñar agentes mucho más robustos, separando claramente:

- el razonamiento del modelo;
- las herramientas externas;
- las decisiones críticas que requieren validación humana.

Además, facilita la comunicación con el equipo de IA para definir cuándo una tarea debe resolverse mediante prompting, cuándo mediante un modelo específico y cuándo mediante lógica tradicional de programación.

Caso 3: Sistema multiagente para control de producción industrial

Problema

Una planta industrial desea automatizar el monitoreo de una línea de producción utilizando varios agentes especializados.

Trabajo del AI Agent Developer

Diseña una arquitectura multiagente donde cada agente tiene una responsabilidad específica:

- Un agente analiza imágenes provenientes de cámaras de inspección;
- otro interpreta datos de sensores de producción;
- otro consulta el historial de mantenimiento;
- un agente supervisor recibe toda esa información, coordina las respuestas y decide si la línea debe continuar funcionando, generar una alarma o detener la producción.

Relación con Machine Learning

El AI Agent Developer no desarrolla necesariamente los modelos de visión artificial ni los modelos predictivos utilizados por cada agente, pero debe comprender cómo funcionan para integrarlos correctamente.

En este tipo de soluciones, el verdadero desafío no consiste en entrenar una red neuronal, sino en diseñar la arquitectura que permite que varios modelos, herramientas y sistemas colaboren de manera autónoma para resolver problemas complejos.



AI UX Designer

Objetivo

Diseñar experiencias de usuario donde la Inteligencia Artificial interactúa de forma intuitiva, eficiente y confiable.

Responsabilidades

- Diseñar interfaces conversacionales.
- Definir flujos de interacción.
- Gestionar incertidumbre.
- Diseñar mecanismos de confirmación.
- Explicar decisiones de la IA.
- Reducir errores de interpretación.
- Mejorar confianza del usuario.

Conocimientos

UX

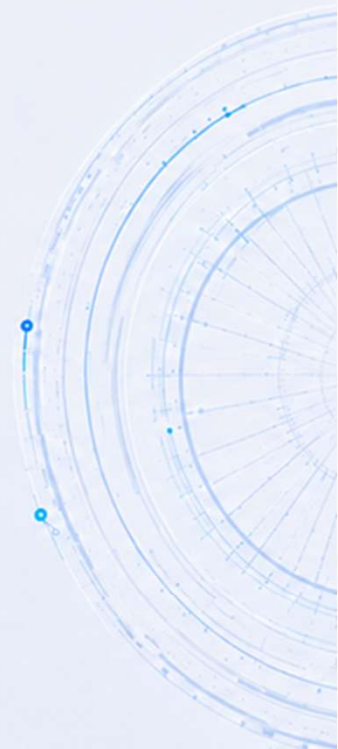
Conversational Design

Human-AI Interaction

Prompt UX

Perfil

Es un diseñador de experiencia de usuario enfocado en aplicaciones inteligentes. Debe comprender tanto principios de UX tradicionales como el comportamiento probabilístico de los modelos de IA, diseñando interfaces que inspiren confianza, reduzcan errores y permitan al usuario interactuar naturalmente con asistentes y agentes inteligentes.





AI Product Manager

Objetivo: Definir productos que incorporen Inteligencia Artificial generando valor para el negocio.

Responsabilidades

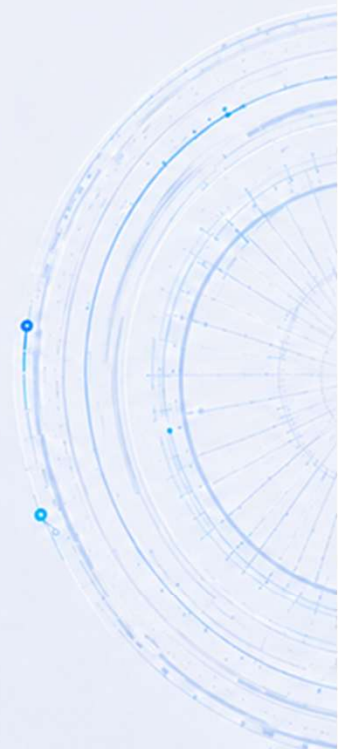
- Detectar oportunidades.
- Priorizar funcionalidades.
- Definir métricas de éxito.
- Coordinar equipos técnicos.
- Validar viabilidad.
- Gestionar roadmap.
- Analizar retorno de inversión.
- Evaluar impacto de la IA en el producto.

Conocimientos

- Producto
- Negocio
- UX
- IA Generativa
- Métricas
- Costos de inferencia

Perfil

Es un perfil que une conocimientos técnicos con visión estratégica de negocio. No desarrolla directamente las soluciones, sino que identifica oportunidades donde la IA puede generar valor, define prioridades y coordina equipos multidisciplinarios para construir productos exitosos. Debe comprender tanto las capacidades como las limitaciones de la Inteligencia Artificial.





AI Consultant

Objetivo

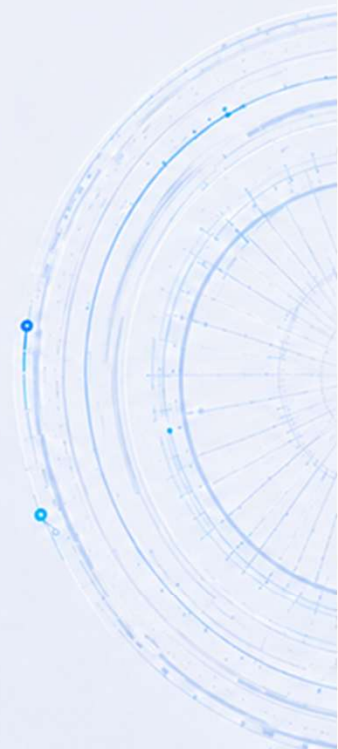
Asesorar organizaciones sobre cómo incorporar IA para resolver problemas concretos de negocio y mejorar procesos.

Responsabilidades

- Analizar procesos existentes.
- Detectar oportunidades de automatización.
- Definir estrategia de adopción.
- Seleccionar tecnologías.
- Estimar costos.
- Diseñar arquitectura.
- Evaluar riesgos.
- Capacitar equipos.

Perfil

Es un rol fuertemente orientado al negocio. Debe comprender tanto las capacidades técnicas de la IA como los procesos operativos de la empresa para proponer soluciones viables y con retorno de inversión.





```
> print("Gracias !")
> print("dudas, consultas...")
> input("Presione una tecla para salir...")
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep. Argentina 786, Sunchoales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383